WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

Multiplexers and Demultiplexers

Lecture 07: combinational circuits that choose and route

Dr. Ali Khudiyev

Fall 2026

CC BY-SA 4.0

Today

01 Combinational circuits

02 The multiplexer

03 Larger multiplexers

04 The demultiplexer

05 Summary

01

Combinational circuits

Combinational circuits

> **Definition (Combinational circuit)**
>
> A circuit whose outputs depend *only* on its present inputs. Give it the same inputs, and it always gives the same outputs: it has no memory of the past.

- On paper, $f(x, y, z) = xy' + x'z$ has no notion of time.
- A real circuit works in time: at step t it sees inputs x_t, y_t, z_t. It is combinational when

$$f(x_t, y_t, z_t) = x_t y'_t + x'_t z_t,$$

  that is, the output at step t depends only on the inputs at step t.
- Circuits with memory, whose outputs also depend on earlier inputs, are *sequential* (Unit 10–14).

Building blocks

A general-purpose computer can be programmed to compute any computable function. Building one takes a few combinational circuits, over and over:

- **multiplexers** and **demultiplexers**: today;
- encoders and decoders `(Unit 8)`;
- adders, comparators and the arithmetic logic unit `(Unit 16–18)`.

> **Remark**
>
> Look again at the DX8 diagram in Lecture 01: the trapezoids labelled MUX choose which value goes into the RAM, the register file and the instruction pointer. By the end of today you can build each of them.

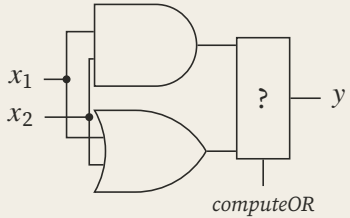
02

The multiplexer

Choosing between two results

We have one circuit that computes $x_1 x_2$ and one that computes $x_1 + x_2$, and a bit *computeOR* that says which result we want:

$$f(x_1, x_2, \mathit{computeOR}) = \begin{cases} x_1 x_2 & \text{if } \mathit{computeOR} = 0, \\ x_1 + x_2 & \text{if } \mathit{computeOR} = 1. \end{cases}$$



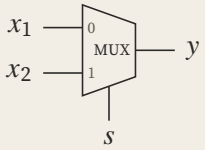
We need a part that passes on one of two inputs, chosen by a bit: like a railway switch, which decides which track a train continues on.

The 2-to-1 multiplexer

Definition (Multiplexer)

A 2-to-1 *multiplexer* (MUX) has two data inputs and one *select* input s:

$$\mathrm{MUX}(x_1, x_2, s) = \begin{cases} x_1 & \text{if } s = 0, \\ x_2 & \text{if } s = 1. \end{cases}$$



The book calls it a 2x1 MUX: 2 inputs, 1 output.

Our function is now one MUX:
$f(x_1, x_2, \mathit{computeOR}) = \mathrm{MUX}(x_1 x_2,\ x_1 + x_2,\ \mathit{computeOR})$.

The truth table

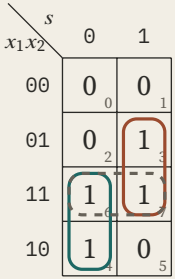
x_1	x_2	s	MUX
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

- Where $s = 0$ the output copies x_1; where $s = 1$ it copies x_2.
- The 1s are on rows 3, 4, 6 and 7. The canonical sum of products (Lecture 04) is

$$x_1' x_2 s + x_1 x_2' s' + x_1 x_2 s' + x_1 x_2 s.$$

- Four products of three literals. Can it be shorter?

Simplifying with a Karnaugh map

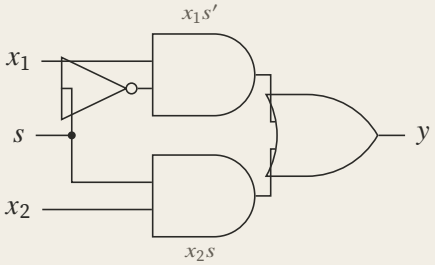


- Teal: $x_1 = 1$, $s = 0$. Product: $x_1 s'$.
- Brick: $x_2 = 1$, $s = 1$. Product: $x_2 s$.
- Dashed: $x_1 x_2$ is prime but not essential. It is the consensus term (Lecture 03).

$$\mathrm{MUX}(x_1, x_2, s) = x_1 s' + x_2 s$$

The book reaches the same result with the laws, ending with a case analysis on $x_1 x_2$.

Inside the multiplexer



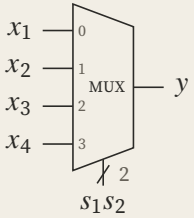
- Each AND gate is a door: $x \cdot 1 = x$ (identity) lets x through, $x \cdot 0 = 0$ (domination) shuts it.
- $s = 0$ opens the top door, so $y = x_1$; $s = 1$ opens the bottom one, so $y = x_2$.
- The OR passes on whichever door is open.

03

Larger multiplexers

The 4-to-1 multiplexer

s_1	s_2	y
0	0	x_1
0	1	x_2
1	0	x_3
1	1	x_4

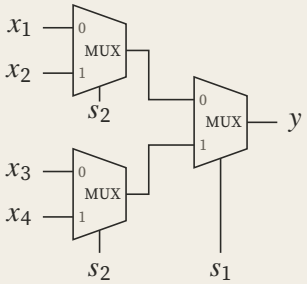


- Four data inputs need two select bits.
- The select bits, read as a binary number, say which input passes.

As a sum of products, each input is ANDed with the minterm of its select value:

$$y = s_1' s_2' \, x_1 + s_1' s_2 \, x_2 + s_1 s_2' \, x_3 + s_1 s_2 \, x_4.$$

Building it from 2-to-1 multiplexers



- The first column picks within each pair, using s_2: x_1 or x_2, and x_3 or x_4.
- The second picks between the pairs, using s_1.
- Check $s_1 s_2 = 01$: the first column passes x_2 and x_4; then $s_1 = 0$ picks x_2, as in the table.

Both s_2 labels are the same wire. The lower select bit goes to the first column, the higher one to the last.

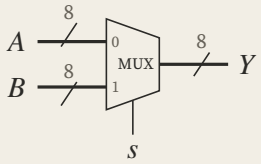
Any number of inputs

- A 2^n-to-1 multiplexer needs n select bits.
- As a tree of 2-to-1 multiplexers it has n columns and $2^{n-1} + \cdots + 2 + 1 = 2^n - 1$ of them.

Inputs	2	4	8
Select bits	1	2	3
2-to-1 multiplexers	1	3	7

> **Remark**
>
> For a number of inputs that is not a power of two, round up: a 3-to-1 multiplexer still needs 2 select bits, and one select value is left unused.

Choosing between whole numbers



- To choose between two 8-bit numbers A and B, use eight 2-to-1 multiplexers, one per bit, all sharing the same s.
- A thick line with a slash and a number is a bus: that many wires side by side.
- This is how the DX8 chooses which value goes into a register or into memory.

A multiplexer can compute any function

- Put the inputs of a function on the select lines, and its truth table on the data inputs. An 8-to-1 multiplexer with data $0, 0, 0, 1, 0, 1, 1, 1$ computes the majority function of Lecture 04.
- Or split on one variable (Shannon's expansion):

$$f(x, y, z) = x' f(0, y, z) + x f(1, y, z) = \mathrm{MUX}(f(0, y, z),\ f(1, y, z),\ x).$$

- For the majority function, $f(0, y, z) = yz$ and $f(1, y, z) = y + z$:

$$maj(x, y, z) = \mathrm{MUX}(yz,\ y + z,\ x).$$

FPGAs, chips whose logic can be reconfigured after they are made, build it from small lookup tables that work this way.

04

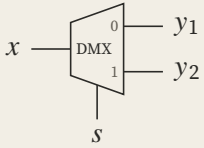
The demultiplexer

The 1-to-2 demultiplexer

The opposite job: one input x, routed to *one of* two outputs. The other output is 0.

x	s	y_1	y_2
0	0	0	0
0	1	0	0
1	0	1	0
1	1	0	1

s	y_1	y_2
0	x	0
1	0	x



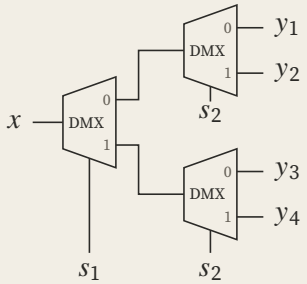
From the table:

$$y_1 = xs', \qquad y_2 = xs.$$

Two AND gates and one NOT: the same doors as in the multiplexer, without the OR.

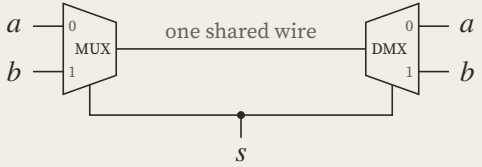
The book calls it a 1x2 DEMUX.

Larger demultiplexers



- A 1-to-4 demultiplexer: s_1 picks a half, then s_2 picks within it.
- y_k is x ANDed with the minterm of its select value: $y_1 = x s_1' s_2'$, …, $y_4 = x s_1 s_2$.
- A 1-to-N demultiplexer needs $\lceil \log_2 N \rceil$ select bits.

One wire, many users



- With the same s on both ends, a and b take turns on one wire. This is where the names come from: *multiplexing* is sending several signals over one line.
- Inside a computer, the same idea lets many parts share one bus.

Coming back: a demultiplexer whose input is fixed at 1 sets exactly one output to 1. That is a decoder (Unit 8).

Try it

1. Write the truth table of a 3-to-1 multiplexer.
2. How many 2-to-1 multiplexers make an 8-to-1 multiplexer?
3. Write y_3 of a 1-to-4 demultiplexer as an expression.
4. Build XOR from one 2-to-1 multiplexer and one NOT gate.

Try it

1. Write the truth table of a 3-to-1 multiplexer.
   Two select bits: $s_1 s_2 = 00, 01, 10$ give x_1, x_2, x_3; the value 11 is unused (a don't-care).
2. How many 2-to-1 multiplexers make an 8-to-1 multiplexer?
3. Write y_3 of a 1-to-4 demultiplexer as an expression.
4. Build XOR from one 2-to-1 multiplexer and one NOT gate.

Try it

1. Write the truth table of a 3-to-1 multiplexer.
   Two select bits: $s_1 s_2 = 00, 01, 10$ give x_1, x_2, x_3; the value 11 is unused (a don't-care).
2. How many 2-to-1 multiplexers make an 8-to-1 multiplexer?
   Seven, in three columns: $4 + 2 + 1$.
3. Write y_3 of a 1-to-4 demultiplexer as an expression.

4. Build XOR from one 2-to-1 multiplexer and one NOT gate.

Try it

1. Write the truth table of a 3-to-1 multiplexer.
   Two select bits: $s_1 s_2 = 00, 01, 10$ give x_1, x_2, x_3; the value 11 is unused (a don't-care).
2. How many 2-to-1 multiplexers make an 8-to-1 multiplexer?
   Seven, in three columns: $4 + 2 + 1$.
3. Write y_3 of a 1-to-4 demultiplexer as an expression.
   $y_3 = x s_1 s_2'$: select value 10.
4. Build XOR from one 2-to-1 multiplexer and one NOT gate.

Try it

1. Write the truth table of a 3-to-1 multiplexer.
   Two select bits: $s_1 s_2 = 00, 01, 10$ give x_1, x_2, x_3; the value 11 is unused (a don't-care).
2. How many 2-to-1 multiplexers make an 8-to-1 multiplexer?
   Seven, in three columns: $4 + 2 + 1$.
3. Write y_3 of a 1-to-4 demultiplexer as an expression.
   $y_3 = x s_1 s_2'$: select value 10.
4. Build XOR from one 2-to-1 multiplexer and one NOT gate.
   $x \oplus y = \text{MUX}(y,\ y',\ x)$: if $x = 0$ pass y, if $x = 1$ pass y'.

05

Summary

Summary

- A combinational circuit's outputs depend only on its present inputs.
- A 2-to-1 multiplexer passes on x_1 or x_2, chosen by s: $\text{MUX}(x_1, x_2, s) = x_1 s' + x_2 s$.
- A 2^n-to-1 multiplexer has n select bits; it can be built as a tree of $2^n - 1$ 2-to-1 multiplexers. One per bit chooses between whole numbers.
- A multiplexer can compute any function from its truth table.
- A 1-to-2 demultiplexer routes x to one of two outputs: $y_1 = xs'$, $y_2 = xs$.

Before the next lecture

- Read the beginning of Chapter 4 and Sections 4.1.1 and 4.1.2 of the book.
- In Logisim-evolution, build a 2-to-1 multiplexer from gates, then a 4-to-1 multiplexer from three of yours. Test it on every select value.
- Build a 1-to-4 demultiplexer, and compare it with the Demultiplexer in the Plexers library.

> **Next lecture** (Unit 8)
>
> Encoders, priority encoders and decoders.